Lab07 – Temperature Sensor

I2C

1. Cos'è I²C?

I²C è un **protocollo di comunicazione seriale** usato per far comunicare un **microcontrollore** con una o più periferiche. La cosa importante è che, a differenza di USART, **I²C permette di collegare più dispositivi usando solamente due fili**.

2. I due fili: SDA e SCL

Linea	Significato	Funzione
SDA	Serial Data	Trasporta i dati
SCL	Serial Clock	Trasporta il clock

A differenza di una comunicazione seriale UART, **non hai TX e RX separati**. SDA è una linea bidirezionale.

3. Perché servono solo due fili?

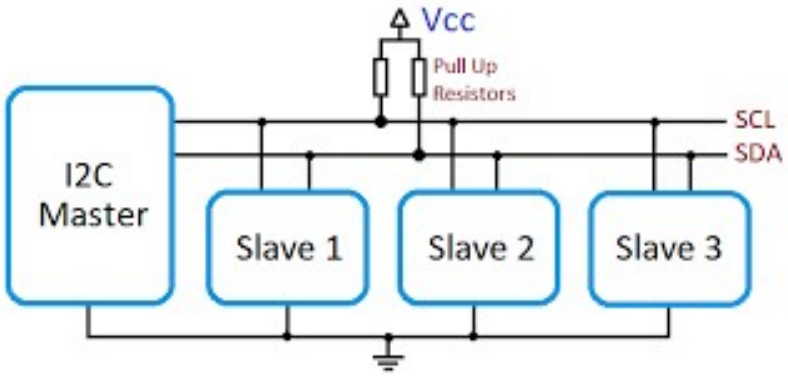
Perché il **clock è generato dal dispositivo Master**.
In una configurazione tipica:

- STM32 → **Master**
- sensore → **Slave**

4. Master e Slave

Questo è uno dei concetti fondamentali di I²C.

- **Master:** È il dispositivo che **inizia la comunicazione e genera il clock**.
- **Slave:** È il dispositivo che **risponde al Master**.



5. Come vengono distinti gli Slave?

Ogni dispositivo I²C ha un **indirizzo**.
Ad esempio:

- Sensor 1 → address 0x48
- Sensor 2 → address 0x68

Il Master può quindi dire: "Voglio comunicare con il dispositivo 0x68." e solo quello risponde.

6. Pull-up: una cosa molto importante

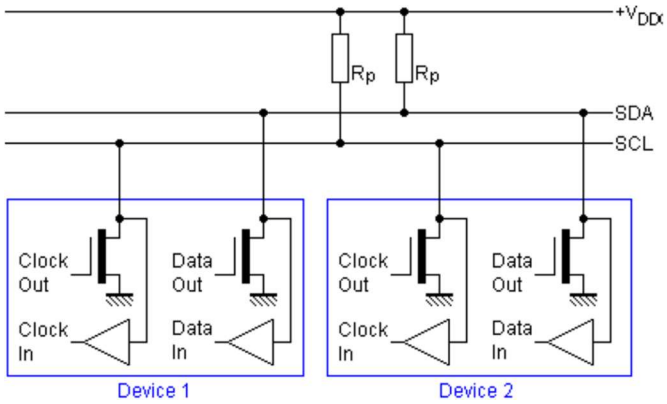
Le linee **SDA e SCL non vengono normalmente portate HIGH direttamente dal dispositivo**. Sono collegate a delle **resistenze di pull-up**. Questo perché le uscite I²C sono generalmente di tipo **open-drain/open-collector**. In pratica:

- **nessun dispositivo forza la linea HIGH**
- **un dispositivo può però portarla LOW**
- **quando nessuno la porta LOW → la resistenza la mantiene HIGH**

Quindi:

- HIGH → ottenuto tramite pull-up
- LOW → ottenuto dal dispositivo

Questo permette a più dispositivi di condividere le stesse linee senza creare conflitti quando comunicano.



7. Come avviene una comunicazione?

1. **START**
2. **ADDRESS**
3. **R/W**
4. **ACK**
5. **DATA**
6. **ACK**
7. **STOP**



1. **START**: Il Master comunica: "Sto iniziando una comunicazione." Lo fa generando una particolare transizione sulle linee SDA/SCL.
2. **ADDRESS**: Il Master invia l'indirizzo dello Slave. Ad esempio: 0x68
3. **R/W:** Indica se il Master vuole:
   a. **Write** → scrivere sullo Slave
   b. **Read** → leggere dallo Slave
4. **ACK**: Lo Slave risponde con un **ACK (Acknowledgement)**. Significa sostanzialmente: "Ho ricevuto correttamente."
5. **DATA**: A questo punto vengono trasferiti i dati. Dopo ogni byte può esserci un ACK.
6. **STOP:** Il Master comunica: "La comunicazione è terminata."

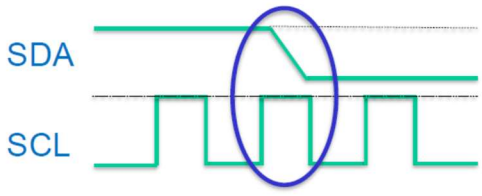


Figura 1: START

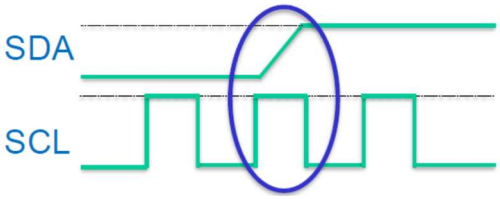
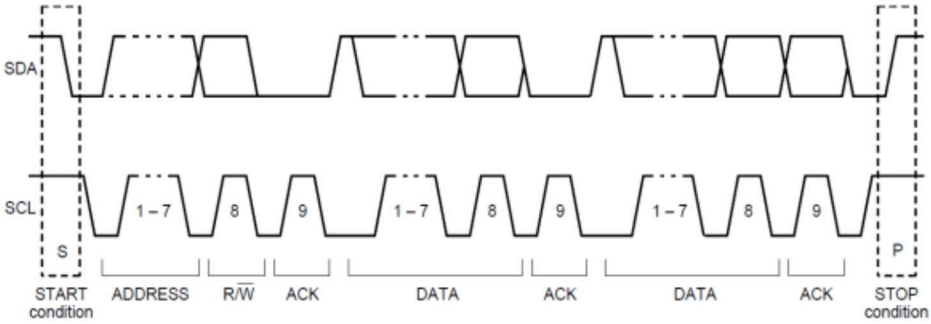


Figura 2: STOP



8. I²C vs UART

	I²C	UART/USART
Fili principali	SDA + SCL	TX + RX
Clock	condiviso	normalmente non condiviso
Master/Slave	Sì	Non nello stesso senso
Più dispositivi	Sì	Non direttamente
Indirizzi	Sì	No
Comunicazione	bidirezionale	bidirezionale
Uso tipico	sensori, EEPROM, RTC	PC, moduli seriali, debug
Pull-up	Sì	No, normalmente

9. Funzioni HAL

1. HAL_I2C_Master_Transmit()

HAL_StatusTypeDef HAL_I2C_Master_Transmit(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t*pData, uint16_t Size, uint32_t Timeout);

Permette allo **STM32 (Master) di inviare dati a un dispositivo I²C (Slave)**. La funzione si occupa automaticamente di gestire la comunicazione I²C, compresi **START, indirizzo, trasferimento dei dati, ACK e STOP**.

Parametro	Significato
hi2c	Handle della periferica I²C utilizzata
DevAddress	Indirizzo del dispositivo Slave
pData	Puntatore ai dati da trasmettere
Size	Numero di byte da trasmettere
Timeout	Tempo massimo di attesa

```
uint8_t data = 0x25;
HAL_I2C_Master_Transmit(
    &hi2c1,      // I2C1
    0x48 << 1,   // indirizzo dispositivo
    &data,       // dati da inviare
    1,           // 1 byte
    1000         // timeout = 1000 ms
);
```

Attenzione a DevAddress: Con le HAL STM32, **l'indirizzo del dispositivo viene normalmente passato nella forma a 8 bit**, quindi per un indirizzo I²C a 7 bit come 0x48 si usa `0x48 << 1` ottenendo 0x90.
Il bit meno significativo viene utilizzato dalla comunicazione per distinguere **Read/Write**.
Quindi è molto comune trovare:

```
#define SENSOR_ADDR (0x48 << 1)

HAL_I2C_Master_Transmit(&hi2c1,
                        SENSOR_ADDR,
                        &data,
                        1,
                        1000);
```

2. HAL_I2C_Master_Receive()

HAL_StatusTypeDef HAL_I2C_Master_Receive(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t*pData, uint16_t Size, uint32_t Timeout);

Permette allo **STM32 (Master) di ricevere dati da un dispositivo I²C (Slave)**. Quindi la direzione dei dati è opposta. I dati ricevuti vengono memorizzati nella memoria indicata da pData.

Parametro	Significato
hi2c	Handle della periferica I²C
DevAddress	Indirizzo dello Slave
pData	Buffer in cui salvare i dati ricevuti
Size	Numero di byte da ricevere
Timeout	Tempo massimo di attesa

La differenza fondamentale è:

- **Transmit → pData contiene i dati DA INVIARE**
- **Receive → pData indica DOVE SALVARE i dati RICEVUTI**

Esempio
Supponiamo di voler ricevere 2 byte dal dispositivo 0x48.

```
uint8_t data[2];

HAL_I2C_Master_Receive(
    &hi2c1,      // I2C1
    0x48 << 1,   // indirizzo dispositivo
    data,        // buffer dove salvare i dati
    2,           // ricevi 2 byte
    1000         // timeout = 1000 ms
);
```

Dopo l'esecuzione `data[0]` conterrà il primo byte ricevuto e `data[1]` il secondo.

pData e il simbolo &

- **Trasmissione**: Se scrivi `uint8_t data = 0x25;` e vuoi **trasmettere** data, passi `&data` perché la funzione vuole un **puntatore** alla memoria contenente i dati.
- **Ricezione**: Invece, se hai `uint8_t data[2];` per ricevere i dati puoi scrivere semplicemente `data` perché il nome di un array viene utilizzato come indirizzo del primo elemento.

3. HAL_I2C_Mem_Write()

```
HAL_StatusTypeDef HAL_I2C_Mem_Write(
  I2C_HandleTypeDef *hi2c,
  uint16_t DevAddress,
  uint16_t MemAddress,
  uint16_t MemAddSize,
  uint8_t *pData,
  uint16_t Size,
  uint32_t Timeout
);
```

Permette allo **STM32 (Master) di scrivere uno o più byte in uno specifico registro interno di un dispositivo I²C (Slave)**. La differenza rispetto a HAL_I2C_Master_Transmit() è proprio questa:

- Master_Transmit → invio semplicemente dei byte
- Mem_Write → invio dei byte a uno specifico REGISTRO

Parametro	Significato
hi2c	Handle della periferica I²C
DevAddress	Indirizzo dello Slave
MemAddress	Indirizzo del registro interno
MemAddSize	Dimensione dell'indirizzo del registro
pData	Dati da scrivere
Size	Numero di byte da scrivere
Timeout	Tempo massimo di attesa

Usala principalmente quando devi **configurare un sensore o una periferica I²C scrivendo nei suoi registri**.
Per esempio, potresti voler modificare:

- modalità di funzionamento
- frequenza di campionamento
- configurazione di interrupt
- range di misura
- registri di controllo

Esempio
Supponiamo di avere:
Indirizzo sensore = 0x48
Registro CONFIG = 0x01
Valore da scrivere = 0x25
Possiamo fare:

```
uint8_t data = 0x25;

HAL_I2C_Mem_Write(
```

```
    &hi2c1,              // I2C1
    0x48 << 1,           // indirizzo dispositivo
    0x01,                // indirizzo registro
    I2C_MEMADD_SIZE_8BIT, // registro di 8 bit
    &data,               // dato da scrivere
    1,                   // 1 byte
    1000                 // timeout: 1000 ms
);
```

4. HAL_I2C_Mem_Read()

```
HAL_StatusTypeDef HAL_I2C_Mem_Read(
  I2C_HandleTypeDef *hi2c,
  uint16_t DevAddress,
  uint16_t MemAddress,
  uint16_t MemAddSize,
  uint8_t *pData,
  uint16_t Size,
  uint32_t Timeout
);
```

Permette allo **STM32 di leggere uno o più byte da uno specifico registro interno di un dispositivo I²C**.
È quindi l'operazione inversa rispetto a HAL_I2C_Mem_Write()

Parametro	Significato
hi2c	Handle della periferica I²C
DevAddress	Indirizzo dello Slave
MemAddress	Indirizzo del registro da leggere
MemAddSize	Dimensione dell'indirizzo del registro
pData	Buffer in cui salvare i dati ricevuti
Size	Numero di byte da leggere
Timeout	Tempo massimo di attesa

È probabilmente **una delle funzioni che userai più spesso con i sensori I²C**. Usala quando devi leggere un dato contenuto in un registro, per esempio:

- temperatura
- accelerazione
- pressione
- stato del sensore
- ID del dispositivo
- valori ADC interni

Esempio
Supponiamo che:

- Sensore → 0x48
- Registro → 0x02

e che il registro contenga un byte di temperatura.

```
uint8_t temperature;

HAL_I2C_Mem_Read(
```

```
    &hi2c1,              // I2C1
    0x48 << 1,           // indirizzo dispositivo
    0x02,                // registro da leggere
    I2C_MEMADD_SIZE_8BIT,// registro di 8 bit
    &temperature,        // dove salvare il dato
    1,                   // 1 byte
    1000                 // timeout
);
```

Dopo l'esecuzione `temperature` conterrà il valore letto dal registro 0x02.

Esempio con più byte

Supponiamo invece che il sensore abbia due registri consecutivi:

- 0x02 → DATA_L
- 0x03 → DATA_H

e vogliamo leggere entrambi.

Possiamo fare:

```
uint8_t data[2];

HAL_I2C_Mem_Read(
    &hi2c1,
    0x48 << 1,
    0x02,
    I2C_MEMADD_SIZE_8BIT,
    data,
    2,
    1000
);
```

Dopo la lettura:

- `data[0]` → contenuto di 0x02
- `data[1]` → contenuto di 0x03

I2C_MEMADD_SIZE_8BIT Indica **quanti bit sono necessari per rappresentare l'indirizzo del registro**.

Un dispositivo potrebbe avere registri indirizzati con:

- 8 bit: I2C_MEMADD_SIZE_8BIT
- 16 bit: I2C_MEMADD_SIZE_16BIT

MemAddSize **non indica la dimensione del dato che vuoi leggere/scrivere**. Indica la dimensione dell'**indirizzo del registro**.

Sensore Interno di Temperatura (LM75/LM75B)

Il **sensore di temperatura interno presente sulla board** è un **LM75/LM75B**, collegato allo STM32 tramite **I²C**. Ci sono infatti batch diversi della board, quindi può esserci LM75 oppure LM75B.

1. Cos'è LM75 / LM75B?

L'**LM75/LM75B è un sensore digitale di temperatura**. La cosa importante è che **non devi usare l'ADC dello STM32 per misurare direttamente una tensione**: il sensore misura la temperatura, la converte internamente in un valore digitale e lo rende disponibile attraverso l'interfaccia **I²C**.

2. Come comunica con lo STM32?

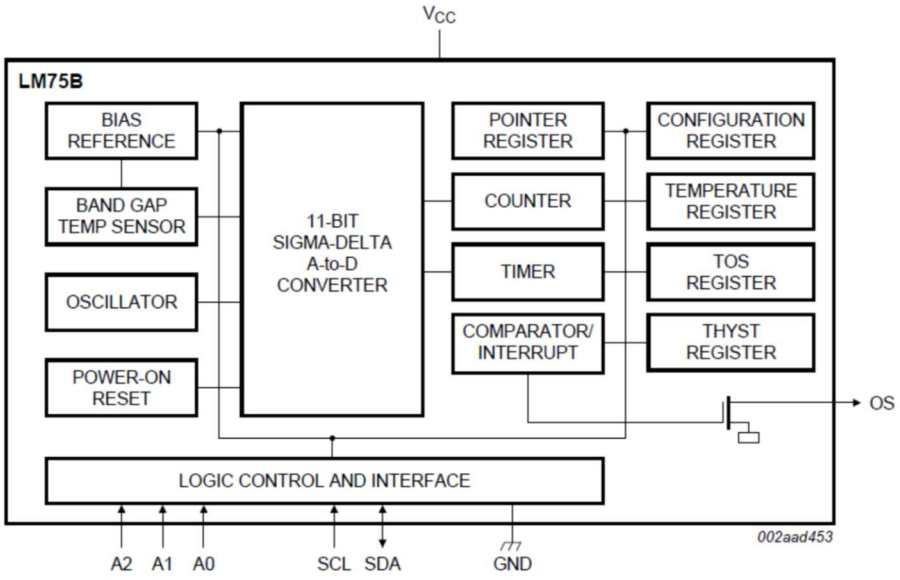
Il sensore è quindi uno **Slave I²C**, mentre lo STM32 è il **Master**.
Il sensore ha un indirizzo I²C di **7 bit** 0b1001000, che corrisponde a 0x48.

```
#define LM75B_ADDR (0b1001000 << 1)
```

3. Il Temperature Register

Il sensore possiede diversi registri interni.
Quello che ci interessa per misurare la temperatura è il **Temperature Register**
Address = 0x00.

Questo registro contiene **il risultato della conversione della temperatura**.



4. Come è rappresentata la temperatura?

Qui c'è una differenza tra **LM75** e **LM75B**. Il sensore utilizza una rappresentazione in **complemento a due**.

- **LM75:**
  - **9 bit** di informazione
  - risoluzione di **0,5 °C**
- **LM75B:**
  - **11 bit** di informazione
  - risoluzione di **0,125 °C**

Entrambi hanno un range -55 °C → +125 °C.

	MSByte								LSByte							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
LM75B	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	X	X	X	X	X
LM75	D8	D7	D6	D5	D4	D3	D2	D1	D0	X	X	X	X	X	X	X

Esempio: 21,875 °C

Questo esempio è riportato direttamente nelle slide ed è molto utile per capire la rappresentazione.
Per: T = 21,875 °C il registro della temperatura contiene 0001010111100000, cioè:

- MSByte = 00010101
- LSByte = 11100000

Quindi lo STM32 può ricevere:

```
uint8_t temp[2];
```

con:

```
temp[0] = 00010101
temp[1] = 11100000
```

e poi ricostruire il valore completo.

5. Come lo leggiamo con STM32?

La funzione adatta è HAL_I2C_Mem_Read() perché abbiamo:

- un **Device Address**
- un **Memory/Register Address**
- dei **dati da leggere**

```
uint8_t temp[2];

HAL_I2C_Mem_Read(
    &hi2c1,
    LM75B_ADDR,
    0x00,
    I2C_MEMADD_SIZE_8BIT,
    temp,
    2,
    HAL_MAX_DELAY
);
```

A questo punto:

- `temp[0]` → MSByte
- `temp[1]` → LSByte

6. Perché uint8_t per il buffer?

Il buffer che riceve i dati dall'I²C deve essere `uint8_t temp[2];` e **non** `int8_t temp[2];`
Il motivo è legato all'**estensione del segno** quando successivamente convertiamo i byte in un tipo più grande. Se un valore è negativo e viene memorizzato in int8_t, quando viene convertito in int16_t il compilatore estende il bit di segno, riempiendo i bit aggiuntivi con 1. Con uint8_t, invece, i bit aggiuntivi vengono riempiti con 0.

7. Una cosa interessante: LM75 vs LM75B

Le slide evidenziano una differenza nel modo in cui la lettura interagisce con la conversione della temperatura.

- **LM75:** Se viene effettuata una lettura mentre è in corso una conversione -> la conversione viene fermata e poi riavviata dopo la lettura.
- **LM75B:** La comunicazione I²C e il convertitore Sigma-Delta sono indipendenti. Quindi una comunicazione I²C **non interrompe la conversione in corso** e letture consecutive non impediscono al registro della temperatura di essere aggiornato.

Progetti

Lab07_1a

Lab07_1a: leggi la Temperatura misurata dal **LM75/LM75B** tramite **I2C** e mandala ad un remote terminal ogni 1 secondo. Leggi solo gli 8 MSB (Most Significant Bits). Verifica che la conversione da binario a Celsius funzioni sia per numeri positivi che negativi.

File .ioc

Pin

- PB8 = I2C1_SCL
- PB9 = I2C1_SDA

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled
- SPI1:
  - I2C: I2C

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LM75_ADDR 0b10010000
#define LM75_TEMP_REGISTER 0x00
```

Private Variables

```
uint8_t MSB;
float temp;
```

```
int len;
uint8_t tx_buf[64];
```

Callback Timer2 (1Hz)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){

		HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR, LM75_TEMP_REGISTER, I2C_ME-
MADD_SIZE_8BIT, &MSB, 1, HAL_MAX_DELAY);

		temp = (float)(int8_t)MSB;   // interpreta il byte grezzo come signed solo
qui


		len = sprintf((char*)tx_buf, "Temperature: %.2f °C\r\n", temp);
		HAL_UART_Transmit_DMA(&huart2, tx_buf, len);

	}
}
```

Inizializzazioni (Main, prima del While)

```
HAL_TIM_Base_Start_IT(&htim2);
```

Lab07_1b

Lab07_1b: leggi la Temperatura misurata dal **LM75/LM75B** tramite **I2C** e mandala ad un remote terminal ogni 1 secondo. Leggi tutti i bit. Verifica che la conversione da binario a Celsius funzioni sia per numeri positivi che negativi.

File .ioc

Pin

- PB8 = I2C1_SCL
- PB9 = I2C1_SDA

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:

        - Request: TX (Memory to Peripheral)
        - Priority: High
        - Mode: Normal
    - NVIC:
        - DMA stream6 global interrupt = Enabled
        - USART2 global interrupt = Enabled
- SPI1:
    - I2C: I2C

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LM75_ADDR 0b10010000
#define LM75_TEMP_REGISTER 0x00
```

Private Variables

```
float temp;
int len;
uint8_t tx_buf[64];
```

Funzioni (fuori dal Main)

```
static void LM75B_ReadTemp(uint8_t *out)
{
    uint8_t r1[2], r2[2];

    HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR, LM75_TEMP_REGISTER, I2C_MEMADD_SIZE_8BIT, r1,
2, HAL_MAX_DELAY);
    HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR, LM75_TEMP_REGISTER, I2C_MEMADD_SIZE_8BIT, r2,
2, HAL_MAX_DELAY);

    if (r1[0] != r2[0])
    {
        /* MSByte cambiato tra le due letture -> possibile "torn read",
           scarto entrambe e ne faccio una terza, ora "pulita" */
        HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR, LM75_TEMP_REGISTER, I2C_MEMADD_SIZE_8BIT,
out, 2, HAL_MAX_DELAY);
    }
    else
    {
        /* le due letture concordano -> il dato è coerente, uso la seconda */
        out[0] = r2[0];
        out[1] = r2[1];
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        uint8_t raw_bytes[2];
        int16_t raw;

        LM75B_ReadTemp(raw_bytes);
```

```
        /* raw_bytes[0]=MSB, raw_bytes[1]=LSB, entrambi uint8_t: nessuna
           estensione di segno indesiderata nella combinazione */
        raw = (int16_t)((raw_bytes[0] << 8) | raw_bytes[1]);
        raw >>= 5;                  /* i dati validi sono gli 11 bit alti (D10..D0) */

        temp = raw * 0.125f;        /* LSB = 0.125 °C per l'LM75B */

        len = sprintf((char*)tx_buf, "Temperature: %.3f °C\r\n", temp);
        HAL_UART_Transmit_DMA(&huart2, tx_buf, len);
    }
}
```

Inizializzazioni (Main, prima del While)

```
HAL_TIM_Base_Start_IT(&htim2);
```

Capire se abbiamo un LM75 o un LM75B

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LM75_ADDR 0b10010000
#define LM75_TEMP_REGISTER 0x00
```

Funzione

```
void LM75_DetectVariant(void)
{
    uint8_t buf[2];
    uint8_t lsb_or_mask = 0x00;
    char msg[80];
    int n;
    const int N_SAMPLES = 40;   /* 40 campioni x 500 ms ~= 20 s di test */

    n = sprintf(msg, "\r\n--- LM75/LM75B detection: scalda/raffredda il sensore per 20s
---\r\n");
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, n, HAL_MAX_DELAY);

    for (int i = 0; i < N_SAMPLES; i++)
    {
        HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR, LM75_TEMP_REGISTER, I2C_MEMADD_SIZE_8BIT,
buf, 2, HAL_MAX_DELAY);

        lsb_or_mask |= buf[1];   /* OR cumulativo di tutti i LSByte visti */

        n = sprintf(msg, "MSB=0x%02X LSB=0x%02X  (OR finora=0x%02X)\r\n", buf[0],
buf[1], lsb_or_mask);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, n, HAL_MAX_DELAY);

        HAL_Delay(500);
    }

    /* bit7 del LSByte porta dato (D0) sia per LM75 che per LM75B.
       bit6 e bit5 (D1, D0) portano dato SOLO per LM75B: per l'LM75
       sono sempre 0 via hardware, quindi se anche uno solo dei due
       compare a 1 in una qualsiasi lettura, non può che essere LM75B. */
    if (lsb_or_mask & 0x60)
    {
        n = sprintf(msg, "\r\n=> LM75B rilevato (11 bit, risoluzione 0.125 C)\r\n");
    }
    else
    {
        n = sprintf(msg, "\r\n=> Probabile LM75 (9 bit, risoluzione 0.5 C)");
    }
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, n, HAL_MAX_DELAY);
}
```

Output

--- LM75/LM75B detection: scalda/raffredda il sensore per 20s ---

MSB=0x1B LSB=0x80 (OR finora=0x80)

MSB=0x1B LSB=0x80 (OR finora=0x80)

MSB=0x1B LSB=0x80 (OR finora=0x80)

MSB=0x1B LSB=0x80 (OR finora=0x80)

MSB=0x1B LSB=0x80 (OR finora=0x80)

MSB=0x1C LSB=0x00 (OR finora=0x80)

MSB=0x1C LSB=0x00 (OR finora=0x80)

MSB=0x1C LSB=0x80 (OR finora=0x80)

MSB=0x1C LSB=0x80 (OR finora=0x80)

MSB=0x1C LSB=0x80 (OR finora=0x80)

MSB=0x1C LSB=0x80 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

MSB=0x1D LSB=0x00 (OR finora=0x80)

=> Probabile LM75 (9 bit, risoluzione 0.5 C)

Come si legge l'output

- MSB = 0x1B, 0x1C, 0x1D = **27, 28, 29** in decimale → per un LM75 (9 bit), il MSByte da solo è già la parte intera della temperatura in °C (l'abbiamo verificato nelle risposte precedenti: temp = MSB + 0.5*bit7(LSB)).
- LSB assume **solo due valori** in tutta la sequenza: 0x00 e 0x80. In binario, 0x80 = 1000 0000, cioè solo il bit 7 (D0, peso 0.5 °C) è mai a 1; tutti gli altri bit (6..0) restano sempre a 0.

Ricostruendo la temperatura campione per campione:

- MSB=0x1B LSB=0x80 → 27 + 0.5 = 27.5 °C
- MSB=0x1C LSB=0x00 → 28 + 0.0 = 28.0 °C
- MSB=0x1C LSB=0x80 → 28 + 0.5 = 28.5 °C
- MSB=0x1D LSB=0x00 → 29 + 0.0 = 29.0 °C (poi resta stabile a 29.0 °C)

Quindi durante il test la temperatura è salita in modo pulito e monotono di **1.5 °C** (27.5 → 29.0), a scatti di **esattamente 0.5 °C** — probabilmente hai tenuto il dito sul sensore e poi l'hai tolto, lasciandolo stabilizzare a 29.0 °C.

Perché questo è conclusivo per LM75 (non solo "probabile")

Il messaggio finale del codice era cauto ("probabile LM75, potresti non aver variato abbastanza") perché in generale un OR-mask sempre fermo a 0x80 potrebbe anche voler dire "non ho fatto variare abbastanza la temperatura per vedere i bit fini di un LM75B". Ma qui non è questo il caso: la temperatura è chiaramente variata di 1.5 °C, un salto ben più che sufficiente perché un LM75B (risoluzione 0.125 °C) avrebbe dovuto mostrare, durante la transizione, valori intermedi come 0x20, 0x40, 0x60, 0xA0, 0xC0, 0xE0 (cioè passaggi di 0.125 °C tra 27.5 e 29.0). Il fatto che il LSByte sia passato in modo pulito solo tra 0x00 e 0x80, senza mai toccare nessun altro valore nonostante un'escursione di temperatura reale e significativa, è la firma esatta di un dispositivo a **9 bit / 0.5 °C** di risoluzione.